
Documentação de Projeto

para o sistema

Vitrine Digital

Versão 1.0

Projeto de sistema elaborado pelo(s) aluno(s) Davi Vinícius Barbosa de Oliveira
como parte da disciplina **Projeto de Software**.

04 de junho de 2026

Tabela de Conteúdo

1. Introdução	1
2. Modelos de Usuário e Requisitos	1
2.1 Descrição de Atores	1
2.2 Modelo de Casos de Uso e Histórias de Usuários	1
2.3 Diagrama de Sequência do Sistema e Contrato de Operações	1
3. Modelos de Projeto	1
3.1 Arquitetura	1
3.2 Diagrama de Componentes e Implantação.	2
3.3 Diagrama de Classes	2
3.4 Diagramas de Sequência	2
3.5 Diagramas de Comunicação	2
3.6 Diagramas de Estados	2
4. Modelos de Dados	2

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Davi Vinicius Barbosa de Oliveira	04/06/2026	Criação da estrutura inicial do documento e definição de requisitos.	1.0
Davi Vinicius Barbosa de Oliveira	04/06/2026	Mudança: Inclusão dos diagramas de Casos de Uso e definição da arquitetura do Sistema.	1.1

1. Introdução

Este documento agrega: 1) a elaboração e revisão de modelos de domínio e 2) modelos de projeto para o sistema Vitrine Digital.

A referência principal para a descrição geral do problema, domínio e requisitos do sistema é o documento de especificação que descreve a visão de domínio do sistema.

O sistema Vitrine Digital consiste numa plataforma web interativa voltada para o comércio eletrônico, projetada para conectar consumidores a um catálogo de produtos de forma dinâmica, ágil e eficiente. O propósito deste projeto de software é documentar e modelar os aspectos arquiteturais, os diagramas de comportamento (casos de uso e sequências) e a estrutura de dados da aplicação. Esta documentação serve como alicerce técnico para garantir a escalabilidade e a robustez do sistema, alinhando as necessidades de negócio com as decisões de engenharia de software antes da fase de desenvolvimento.

2. Modelos de Usuário e Requisitos

2.1 Descrição de Atores

Nesta subseção é apresentada a descrição de cada um dos atores que interagem com o sistema Vitrine Digital.

- **Cliente:** Usuário final que utiliza a plataforma web. É responsável por navegar pelo catálogo, pesquisar itens, adicionar produtos ao carrinho de compras, realizar o checkout (finalização do pedido) e acompanhar o status de suas compras.
- **Administrador:** Usuário com privilégios de gestão no sistema. Interage com a plataforma para gerenciar o catálogo de produtos (operações de criação, leitura, atualização e exclusão - CRUD), controlar os níveis de estoque, organizar categorias e monitorar as vendas realizadas.
- **Gateway de Pagamento (Ator Externo):** Sistema de terceiros integrado à plataforma. Atua recebendo os dados de cobrança gerados no checkout, processando a transação financeira (via cartão de crédito, Pix, boleto, etc.) e retornando o status do pagamento (aprovado, recusado, pendente) para o sistema Vitrine Digital.
- **API de Logística (Ator Externo):** Serviço externo utilizado para consulta de frete. Interage com o sistema recebendo os dados de CEP e dimensões do pacote para retornar as opções de envio, prazos e valores aplicáveis à compra do Cliente.

2.2 Modelo de Casos de Uso

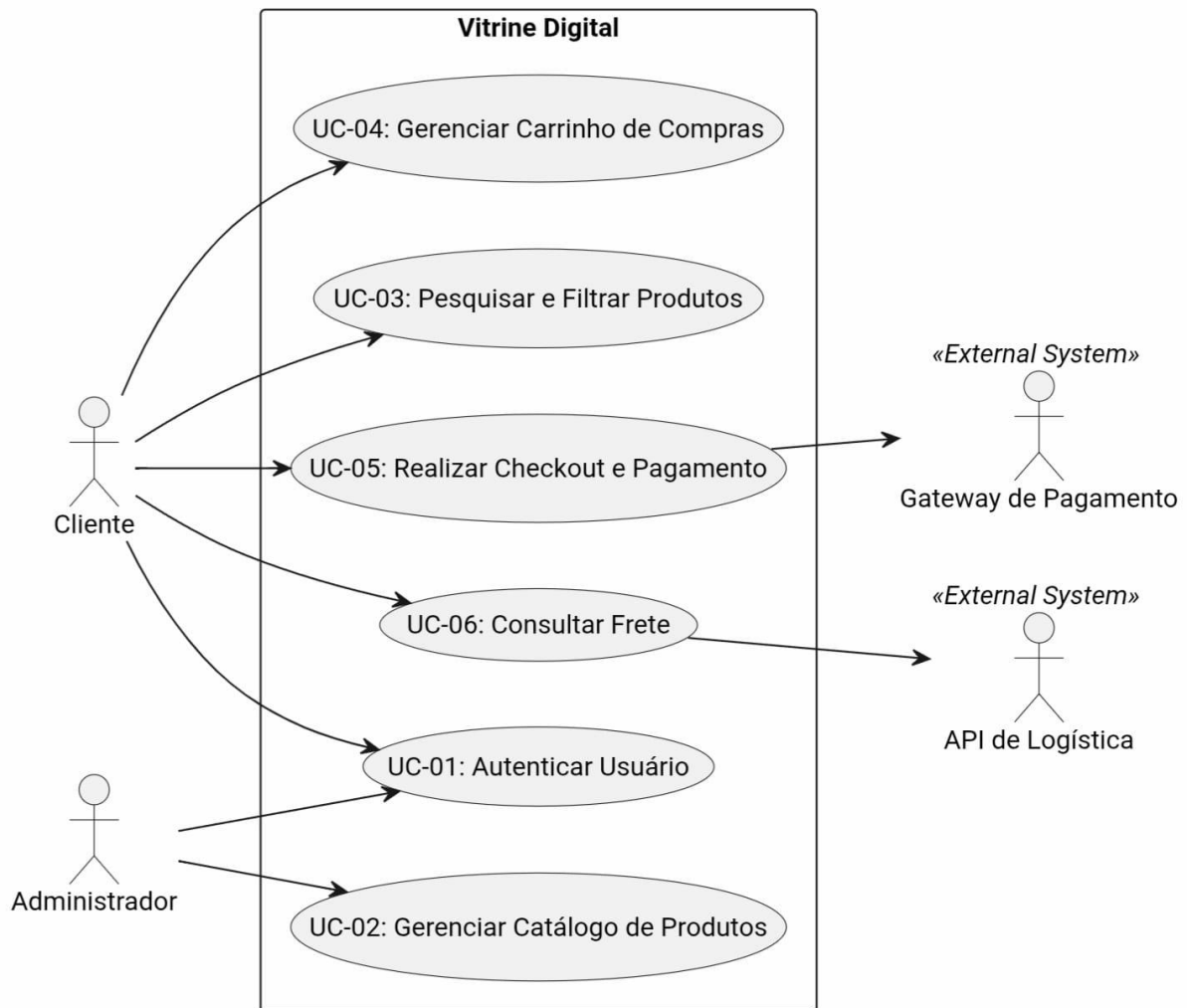
Nesta subseção é apresentado o diagrama de casos de uso do sistema , juntamente com as principais histórias de usuários, utilizando um ID para servir de referência no restante do documento.

Relação de Casos de Uso:

- **UC-01:** Autenticar Usuário
- **UC-02:** Gerenciar Catálogo de Produtos
- **UC-03:** Pesquisar e Filtrar Produtos
- **UC-04:** Gerenciar Carrinho de Compras
- **UC-05:** Realizar Checkout e Pagamento
- **UC-06:** Consultar Frete

Histórias de Usuários Principais:

- **US-01 (Ref. UC-01):** Como Cliente, quero me cadastrar e fazer login no sistema para poder salvar meu histórico de compras e endereços de entrega.
- **US-02 (Ref. UC-02):** Como Administrador, quero adicionar, editar e remover produtos do catálogo para manter o estoque da loja sempre atualizado.
- **US-03 (Ref. UC-04):** Como Cliente, quero poder adicionar diversos produtos ao meu carrinho de compras para realizar o pagamento de todos os itens de uma única vez.
- **US-04 (Ref. UC-05 e UC-06):** Como Cliente, quero informar meu CEP para calcular o frete e, em seguida, pagar com segurança para finalizar minha compra.



2.3 Diagrama de Sequência do Sistema

Nesta subseção são apresentados os contratos de operação para os três principais fluxos de interação do sistema Vitrine Digital.

Formato para cada contrato de operação

Contrato	UC-01 - efetuarLogin
Operação	efetuarLogin(email, senha)
Referências cruzadas	UC-01, US-01
Pré-condições	O usuário já deve possuir um cadastro ativo no sistema com o e-mail informado
Pós-condições	O sistema valida as credenciais informadas, gera um token de acesso seguro (JWT), inicia a sessão do usuário e retorna a confirmação de sucesso junto com o perfil de acesso correspondente

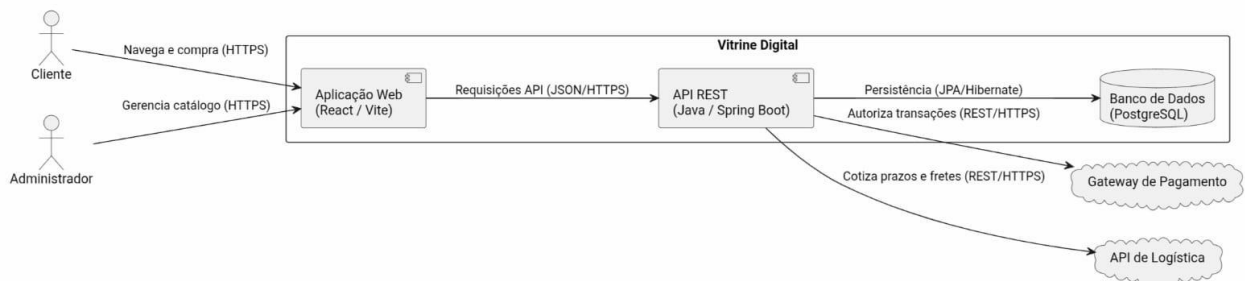
3. Modelos de Projeto

3.1 Arquitetura

O sistema **Vitrine Digital** adota uma arquitetura cliente-servidor distribuída em camadas bem definidas, garantindo o desacoplamento entre a interface de usuário e as regras de negócio. A estrutura segue o modelo de containers para ilustrar a divisão tecnológica e o fluxo de comunicação entre os subsistemas e as integrações externas.

A arquitetura é composta por três camadas principais:

1. **Camada de Apresentação (Front-end):** Uma aplicação de página única (SPA) desenvolvida com React e Vite, responsável pela renderização da interface gráfica, manipulação do estado local do carrinho de compras e interações dinâmicas do usuário.
2. **Camada de Negócio (Back-end):** Uma API REST robusta estruturada com Java e Spring Boot. Essa camada organiza-se internamente nos padrões de projeto *Controller*, *Service* e *Repository*, isolando os endpoints HTTP das regras core de negócio e dos acessos ao banco de dados.
3. **Camada de Persistência (Banco de Dados):** Um banco de dados relacional PostgreSQL encarregado do armazenamento seguro e estruturado dos dados de usuários, produtos, pedidos e logs.



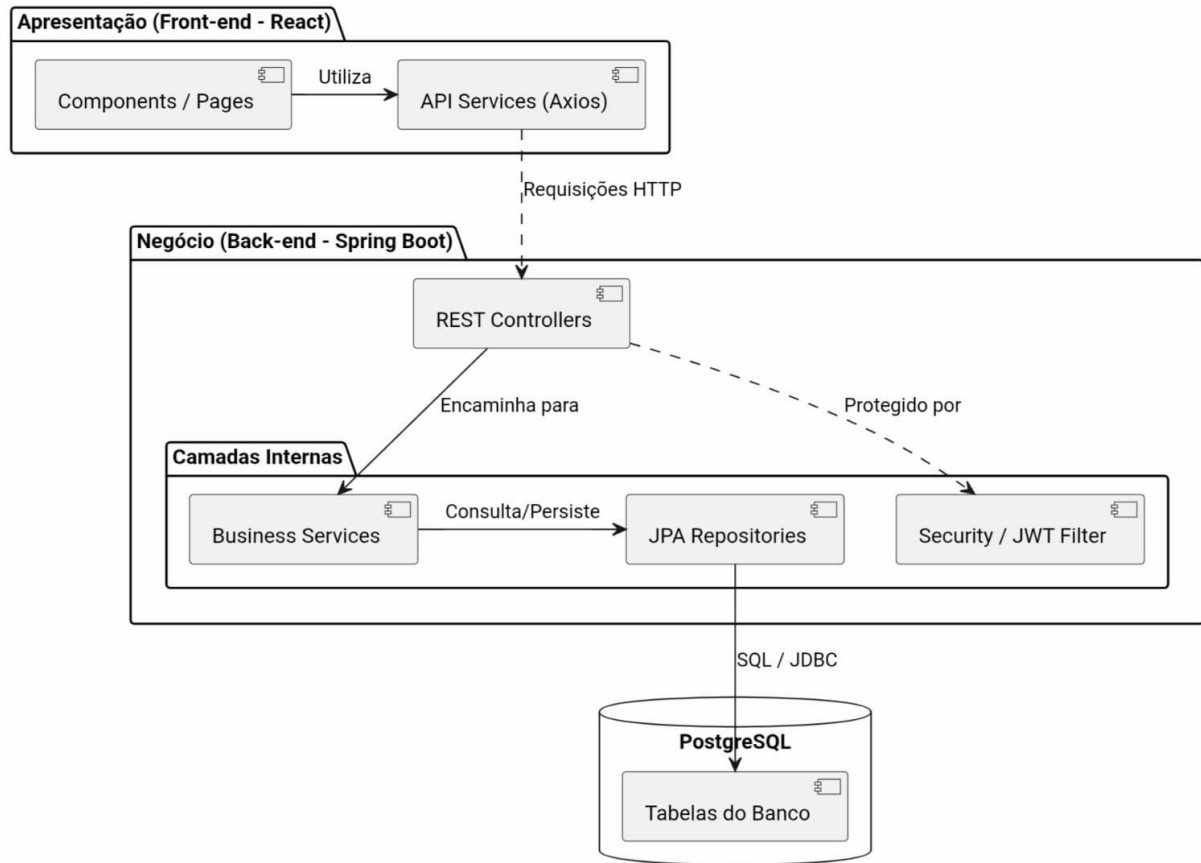
3.2 Diagrama de Componentes e Implantação.

Nesta subseção são apresentados os diagramas de componentes, que ilustram a organização interna dos módulos do sistema, e o diagrama de implantação, que demonstra a distribuição física dos artefatos nos ambientes de execução.

A) Diagrama de Componentes

O diagrama de componentes detalha a estrutura interna do Front-end (React) e do Back-end (Spring Boot). No Front-end, destaca-se a separação entre a interface visual (Pages/Components)

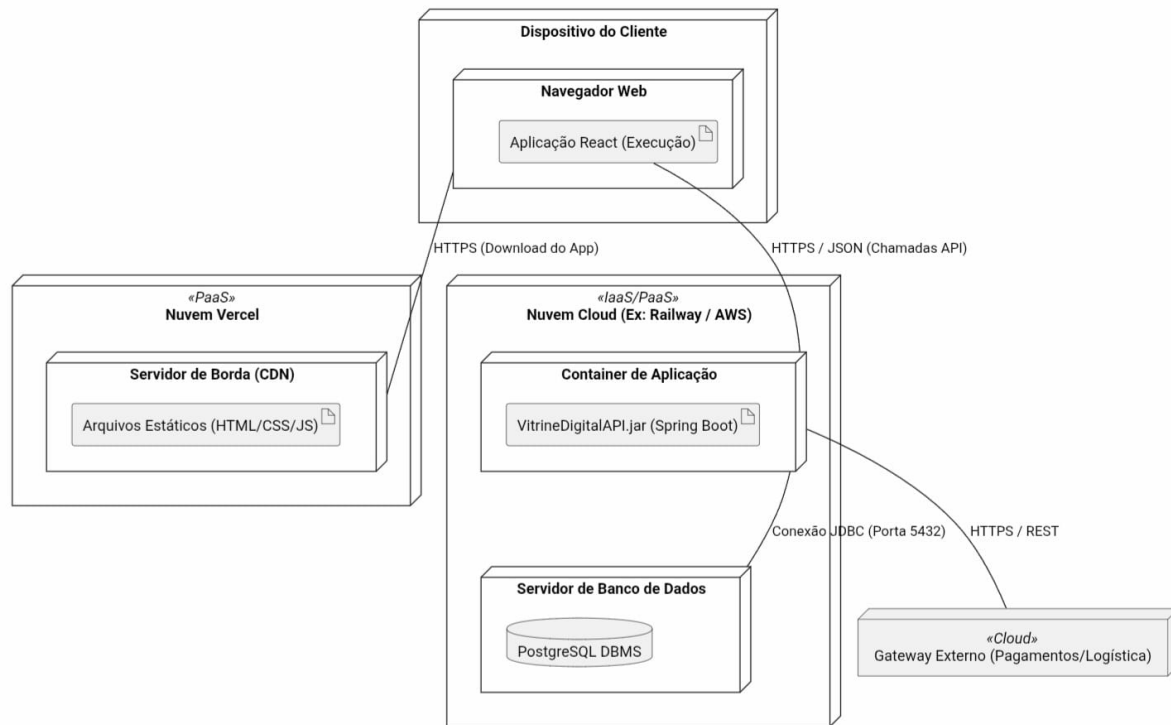
e a comunicação com a API (Services). No Back-end, evidencia-se a divisão clássica de responsabilidades em camadas: os *Controllers* expõem os endpoints, os *Services* executam as regras de negócio e validações, e os *Repositories* gerenciam o mapeamento objeto-relacional com o banco de dados.



B) Diagrama de Implantação

O diagrama de implantação mapeia a topologia física de rede e os nós de infraestrutura onde o sistema **Vitrine Digital** será executado.

- O ambiente do cliente necessita apenas de um navegador web moderno para descarregar os arquivos estáticos.
- A aplicação Front-end está hospedada na plataforma **Vercel**, especializada em distribuição otimizada de aplicações JavaScript/TypeScript.
- A API Back-end e o banco de dados PostgreSQL encontram-se em nós isolados na infraestrutura de nuvem (como **Railway** ou instâncias cloud equivalentes), comunicando-se de forma segura em uma rede privada.

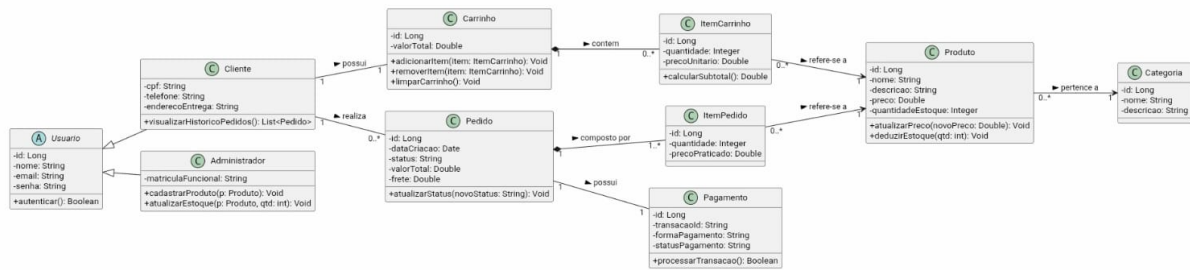


3.3 Diagrama de Classes

O diagrama de classes abaixo representa a estrutura de dados e o modelo de domínio do sistema Vitrine Digital. Ele estabelece as entidades essenciais do negócio, os seus respectivos atributos e comportamentos, além dos relacionamentos fundamentais que garantem a integridade das operações da plataforma.

Destaques da Modelagem:

- **Herança de Utilizadores:** A classe abstrata `Usuario` concentra os atributos comuns de autenticação, sendo especializada pelas subclasses `Cliente` (que realiza compras) e `Administrador` (que gere o catálogo).
- **Ciclo de Compras:** O `Cliente` possui uma relação direta com um `Carrinho` de compras ativo, que é composto por múltiplas instâncias de `ItemCarrinho`. Cada item está associado a um `Produto`.
- **Fechamento do Pedido:** Quando o checkout é concluído, um `Pedido` independente é gerado, composto por linhas de `ItemPedido` (preservando o histórico de preços praticados no momento da compra, mesmo se o produto sofrer alterações futuras) e vinculado a um `Pagamento`.

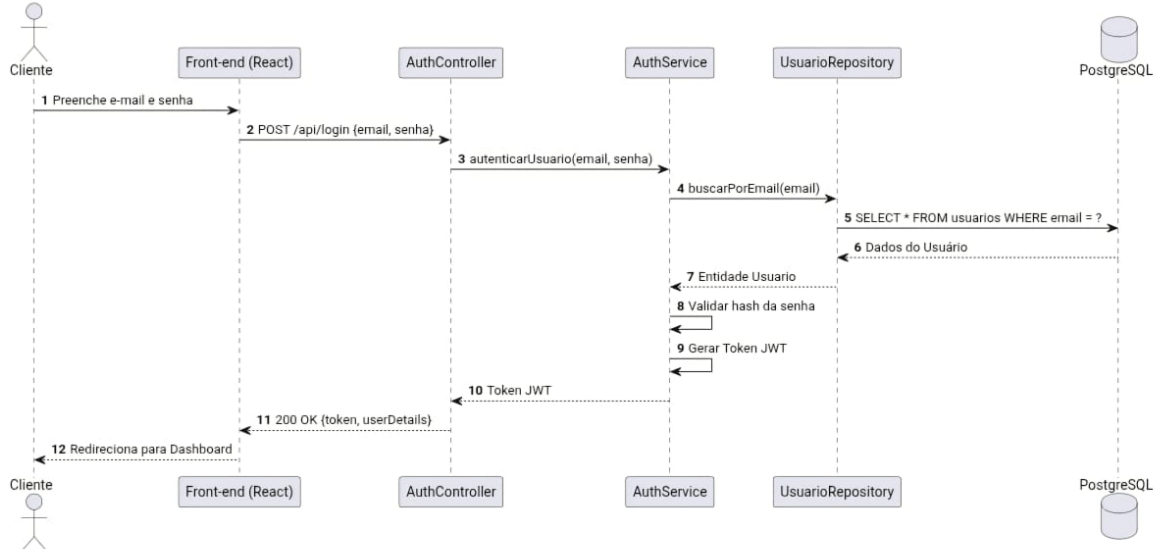


3.4 Diagramas de Sequência

Nesta subseção são apresentados os diagramas de sequência que mapeiam a interação e a troca de mensagens entre os componentes do sistema (Front-end, Controladores, Serviços, Repositórios e Atores Externos) para a realização dos principais casos de uso.

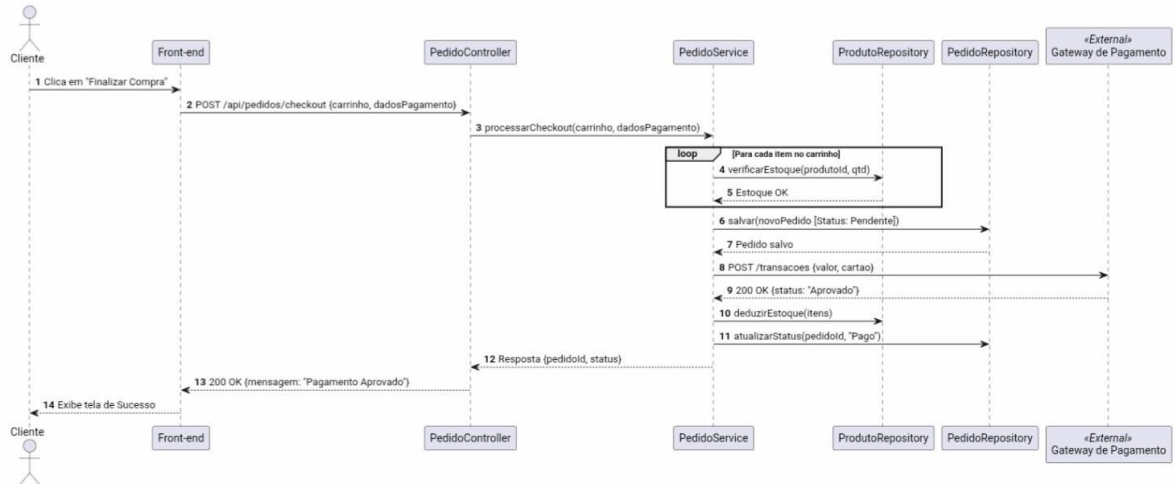
A) Diagrama de Sequência: Autenticar Usuário (UC-01)

Este diagrama ilustra o fluxo de login. O cliente envia as suas credenciais pela interface, a API recebe a requisição, o serviço de negócio valida os dados no banco de dados, gera um token JWT de segurança e o retorna para a sessão do utilizador.



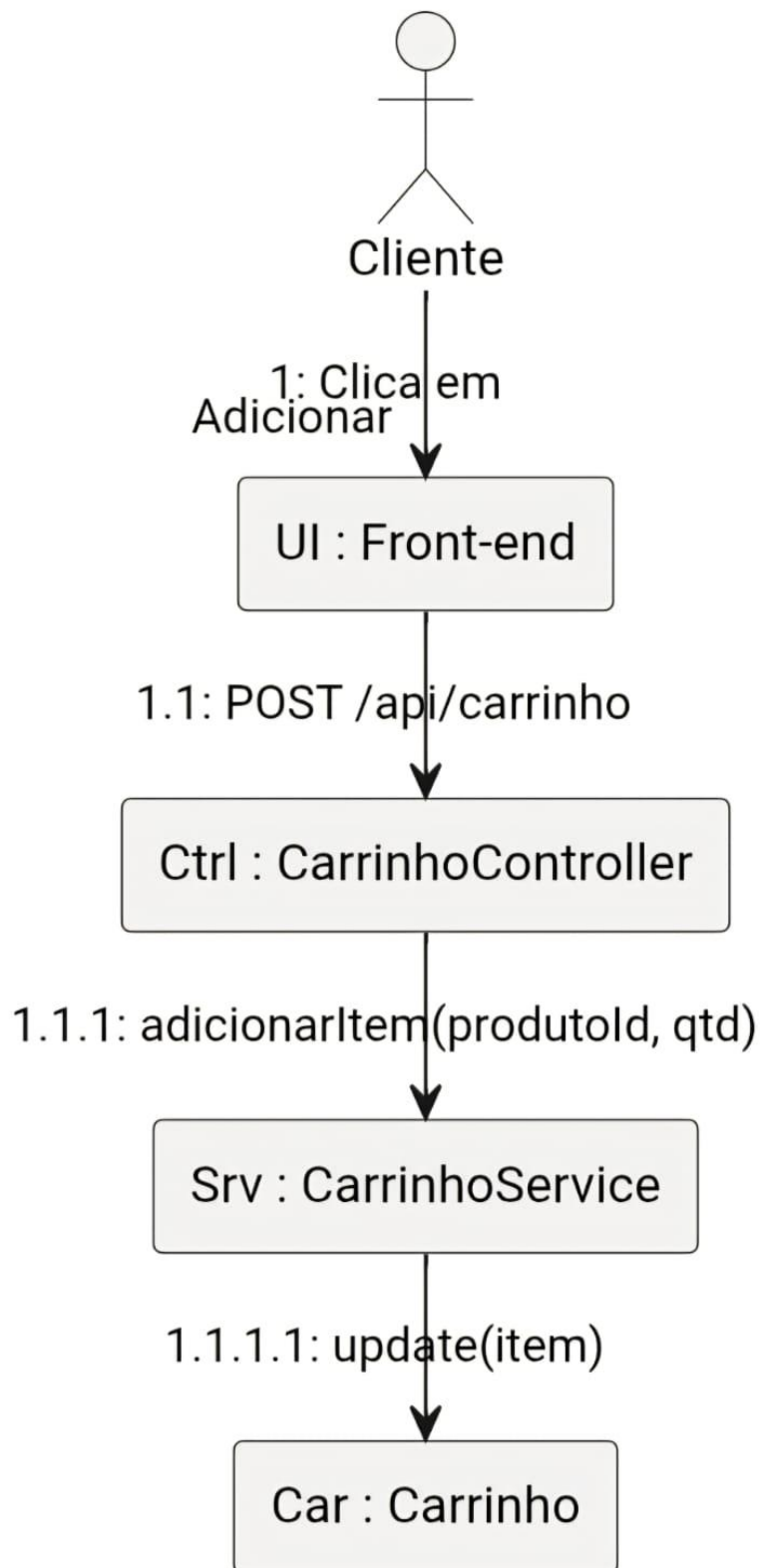
B) Diagrama de Sequência: Realizar Checkout e Pagamento (UC-05)

Este diagrama detalha o processo de finalização de compra. Envolve a criação do pedido interno, a dedução do estoque e a comunicação síncrona com o Gateway de Pagamento externo para autorizar a transação.



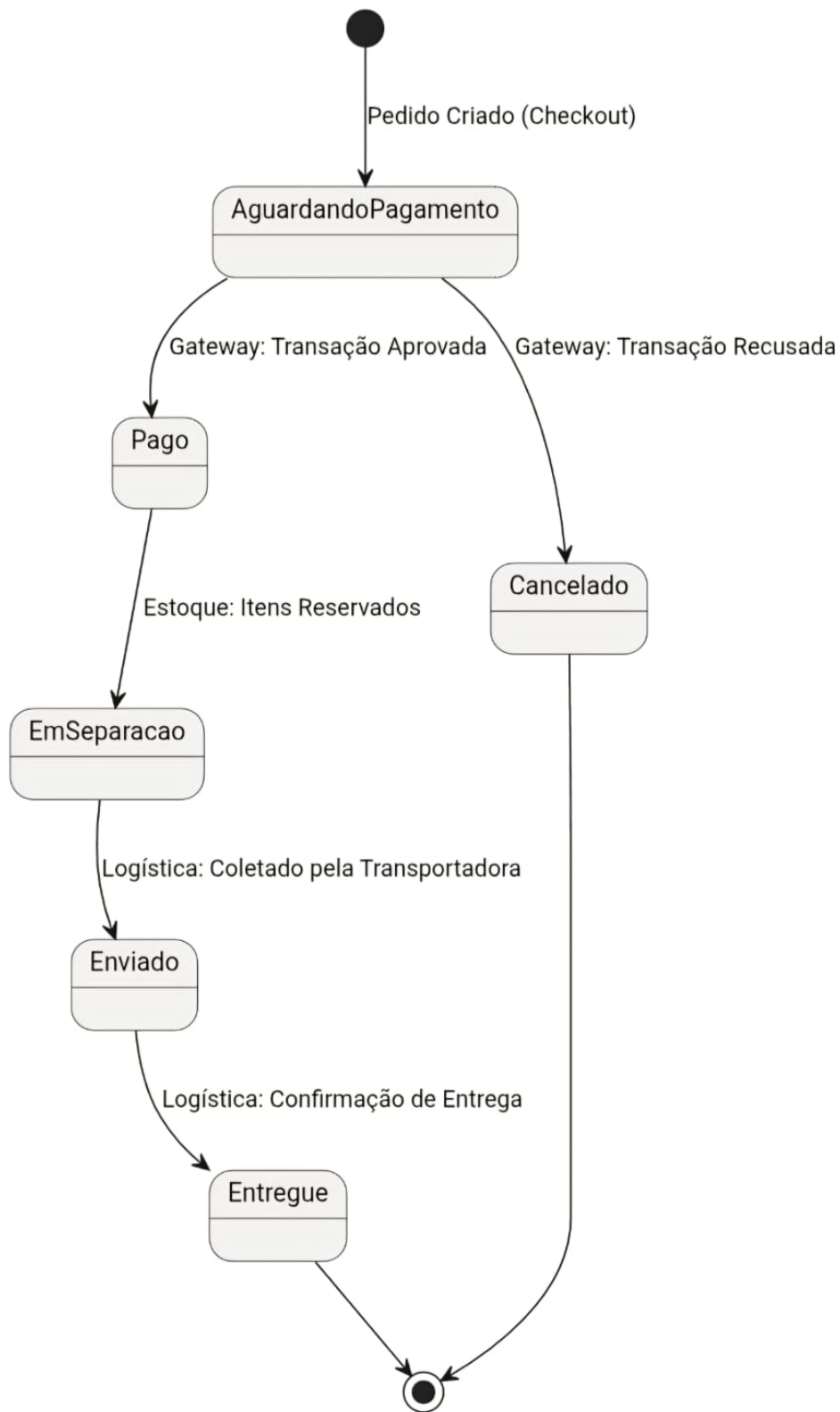
3.5 Diagramas de Comunicação

O diagrama de comunicação (ou colaboração) destaca a organização estrutural dos objetos que enviam e recebem mensagens. Abaixo, modelamos o fluxo da operação de **Adicionar Item ao Carrinho (UC-04)**, enfatizando as ligações entre as instâncias da arquitetura.



3.6 Diagramas de Estados

Este diagrama ilustra o ciclo de vida dinâmico de uma entidade crítica do sistema. Para o **Vitrine Digital**, a entidade com as transições de estado mais complexas é o **Pedido**. O fluxo abrange desde a criação até a entrega ou cancelamento.



4. Modelos de Dados

Esta seção apresenta o esquema relacional do banco de dados (PostgreSQL) e a estratégia de mapeamento objeto-relacional.

Estratégia de Mapeamento (ORM):

O sistema utiliza o padrão JPA (Java Persistence API) com o framework Hibernate para realizar o mapeamento entre os objetos de domínio (camada Java) e as tabelas do banco de dados relacional. As classes do diagrama de domínio (Seção 3.3) recebem anotações como `@Entity`, `@Table`, e `@OneToMany` para automatizar a persistência. A estratégia de geração de chaves primárias será a `GenerationType.IDENTITY` (Auto Incremento delegada ao banco de dados).

Esquema de Banco de Dados (Diagrama ER):

O modelo lógico abaixo ilustra as tabelas principais, suas chaves primárias (PK), chaves estrangeiras (FK) e os tipos de dados básicos.

Código PlantUML - Modelo ER:

